PIQC User Guide

Production Inference Quality Control

Version 1.0.0

January 12, 2026

What is PIQC?

PIQC (Production Inference Quality Control) is a Kubernetes-native tool that automatically discovers and documents AI/ML inference deployments running on your cluster.

Key Features:

- Automatic vLLM deployment discovery
- GPU metrics collection (utilization, memory, temperature)
- Runtime metrics from vLLM API (latency, throughput, cache)
- Multiple output formats (YAML, JSON, Table)
- Works remotely via kubectl or in-cluster

Contents

1 Installation

1.1 Prerequisites

- Python 3.11 or higher
- `kubectl` configured with cluster access
- `poetry` (Python dependency manager)
- Access to a Kubernetes cluster with vLLM deployments

1.2 Download from GitHub

```
# Clone the repository
git clone https://github.com/your-org/ModelSpec.git
cd ModelSpec

# Or download ZIP and extract
# wget https://github.com/your-org/ModelSpec/archive/main.zip
# unzip main.zip
# cd ModelSpec-main
```

1.3 Install Dependencies

PIQC uses Poetry for dependency management:

```
# Install Poetry if not already installed
curl -sSL https://install.python-poetry.org | python3 -

# Install PIQC dependencies
poetry install

# Verify installation
poetry run piqc version
```

Expected output:

```
ModelSpec Introspector v1.0.0
```

2 Quick Start: Testing vLLM Deployments

2.1 Verify Cluster Access

```
# Test Kubernetes connection
poetry run piqc test-connection
```

Expected output:

```
[INFO] Connecting to cluster...
       Context: my-k8s-cluster
       Cluster: my-k8s-cluster
[INFO] Connection successful!
```

2.2 Scan for vLLM Deployments

2.2.1 Basic Table View

```
# Scan all namespaces, show table output
poetry run piqc scan --format table
```

2.2.2 Scan Specific Namespace

```
# Scan only the 'inference' namespace
poetry run piqc scan --namespace inference --format table
```

2.2.3 Full YAML Output

```
# Generate detailed ModelSpec YAML
poetry run piqc scan --namespace inference --format yaml
```

Files are saved to `./output/` by default.

2.3 Collect Runtime Metrics

To collect live vLLM API metrics (latency, throughput, cache):

```
poetry run piqc scan --namespace inference \
    --collect-runtime \
    --format yaml \
    -o ./my-output
```

What you get:

- Request latency percentiles (TTFT, TPOT, E2E)
- Throughput (tokens/sec)
- KV cache usage
- Prefix cache hit rates

3 Testing on Google Cloud Platform (GCP)

3.1 Option 1: Using Cloud Shell (Recommended)

Cloud Shell provides a pre-configured environment with `kubectl` and `python`.

3.1.1 Step 1: Open Cloud Shell

1. Go to https://console.cloud.google.com
2. Click the **Activate Cloud Shell** icon (top-right)
3. Wait for the terminal to load

3.1.2 Step 2: Set Up Your Project

```
# Check current project
gcloud config get-value project

# Set project if needed
gcloud config set project YOUR_PROJECT_ID

# Enable required APIs
gcloud services enable container.googleapis.com
gcloud services enable compute.googleapis.com
```

3.1.3 Step 3: Create GKE Cluster with GPU

```
gcloud container clusters create piqc-test \
  --zone=us-central1-a \
  --machine-type=n1-standard-4 \
  --accelerator=type=nvidia-tesla-t4,count=1 \
  --num-nodes=1 \
  --spot \
  --disk-size=50GB
```

Cost: ~$0.50/hour with Spot instances

3.1.4 Step 4: Install GPU Drivers

```
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/
    container-engine-accelerators/master/nvidia-driver-installer/cos/daemonset-
    preloaded.yaml

# Wait for drivers to install (~2 minutes)
kubectl get pods -n kube-system -w | grep nvidia
```

3.1.5 Step 5: Deploy Test vLLM Server

```
# Create namespace
kubectl create namespace inference

# Deploy TinyLlama on vLLM
cat << 'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-tinyllama
  namespace: inference
  labels:
    app: vllm-tinyllama
    app.kubernetes.io/name: vllm
    framework: vllm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vllm-tinyllama
  template:
    metadata:
```

```
      labels:
        app: vllm-tinyllama
        app.kubernetes.io/name: vllm
        framework: vllm
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:v0.6.3.post1
        args:
          - "--model"
          - "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
          - "--served-model-name"
          - "tinyllama"
          - "--dtype"
          - "float16"
          - "--max-model-len"
          - "1024"
          - "--gpu-memory-utilization"
          - "0.5"
        ports:
          - containerPort: 8000
        resources:
          requests:
            nvidia.com/gpu: 1
            cpu: "2"
            memory: "8Gi"
          limits:
            nvidia.com/gpu: 1
            cpu: "4"
            memory: "12Gi"
---
apiVersion: v1
kind: Service
metadata:
  name: vllm-tinyllama
  namespace: inference
spec:
  selector:
    app: vllm-tinyllama
  ports:
    - port: 8000
      targetPort: 8000
  type: ClusterIP
EOF

# Wait for pod to be ready (~3-5 minutes)
kubectl get pods -n inference -w
```

3.1.6 Step 6: Upload PIQC to Cloud Shell

Method A: Clone from GitHub

```
cd ~
git clone https://github.com/your-org/ModelSpec.git
cd ModelSpec
poetry install
```

Method B: Upload ZIP File

1. Create ZIP on your local machine: `ModelSpec.zip`
2. In Cloud Shell, click menu → **Upload**
3. Select `ModelSpec.zip`
4. Extract and install:

```
cd ~
unzip ModelSpec.zip
cd ModelSpec
poetry install
```

3.1.7 Step 7: Apply RBAC Permissions

```
cd ~/ModelSpec
kubectl apply -f rbac/
```

This grants PIQC permission to exec into pods and list cluster resources.

3.1.8 Step 8: Run PIQC Tests

```
# Basic scan
poetry run piqc scan --namespace inference --format table

# Full scan with runtime metrics
poetry run piqc scan --namespace inference \
    --collect-runtime \
    --format yaml \
    -o ~/piqc-outputs

# View generated file
cat ~/piqc-outputs/inference_vllm-tinyllama.yaml
```

3.1.9 Step 9: Download Results

```
# Create ZIP of outputs
cd ~
zip -r piqc-outputs.zip piqc-outputs/

# Download via Cloud Shell menu
# Click         Download     enter "piqc-outputs.zip"
```

3.1.10 Step 10: Cleanup (Important!)

```
# Delete vLLM deployment
kubectl delete deployment vllm-tinyllama -n inference

# Delete cluster to stop charges
gcloud container clusters delete piqc-test \
  --zone=us-central1-a \
  --quiet
```

4 Using PIQC from Your Local Machine

PIQC can also run from your laptop/workstation using an existing kubeconfig.

4.1 Prerequisites

- Python 3.11+
- `kubectl` installed and configured
- Valid kubeconfig with cluster access
- Poetry installed

4.2 Windows Setup

4.2.1 Install Python

Download Python 3.11+ from https://www.python.org/downloads/

4.2.2 Install kubectl

```
# Using Chocolatey
choco install kubernetes-cli

# Or download from https://kubernetes.io/docs/tasks/tools/
```

4.2.3 Install Poetry

```
# PowerShell
(Invoke-WebRequest -Uri https://install.python-poetry.org -
   UseBasicParsing).Content | python -
```

Add to PATH: `C:\Users\<YourName>\AppData\Roaming\Python\Scripts`

4.2.4 Configure kubectl

Option 1: Download kubeconfig from GKE

```
gcloud container clusters get-credentials CLUSTER_NAME \
  --zone=ZONE \
  --project=PROJECT_ID
```

Option 2: Use provided kubeconfig file

```
# Save kubeconfig to specific location
$env:KUBECONFIG = "C:\path\to\kubeconfig.yaml"
kubectl get nodes
```

4.2.5 Install and Run PIQC

```
# Clone repository
cd C:\Users\YourName\Desktop
git clone https://github.com/your-org/ModelSpec.git
cd ModelSpec
```

```
# Install dependencies
poetry install

# Test connection
poetry run piqc test-connection

# Run scan
poetry run piqc scan --namespace inference \
  --collect-runtime \
  --format yaml \
  -o C:\Users\YourName\Desktop\piqc-outputs
```

4.3 Linux/macOS Setup

4.3.1 Install Prerequisites

```
# macOS (using Homebrew)
brew install python@3.11
brew install kubectl

# Ubuntu/Debian
sudo apt update
sudo apt install python3.11 python3-pip kubectl

# Install Poetry
curl -sSL https://install.python-poetry.org | python3 -
```

4.3.2 Configure kubectl

```
# Get credentials from GKE
gcloud container clusters get-credentials CLUSTER_NAME \
  --zone=ZONE \
  --project=PROJECT_ID

# Or set custom kubeconfig
export KUBECONFIG=/path/to/kubeconfig.yaml
kubectl get nodes
```

4.3.3 Install and Run PIQC

```
# Clone repository
cd ~/Desktop
git clone https://github.com/your-org/ModelSpec.git
cd ModelSpec

# Install dependencies
poetry install

# Run scan
poetry run piqc scan --namespace inference \
  --collect-runtime \
  --format yaml \
  -o ~/piqc-outputs
```

5 Command Reference

5.1 Core Commands

Command	Description
`piqc version`	Show version information
`piqc test-connection`	Verify cluster connectivity
`piqc scan`	Scan cluster for vLLM deployments

5.2 Scan Options

Option	Description
`-namespace, -n`	Scan specific namespace(s)
`-format, -f`	Output format: table, yaml, json
`-output, -o`	Output directory
`-collect-runtime`	Collect vLLM API runtime metrics
`-no-exec`	Skip GPU metrics (faster)
`-verbose, -v`	Enable verbose logging
`-debug`	Enable debug logging
`-workers`	Number of parallel workers
`-combined`	Output single combined file
`-output-piqc`	Generate PIQC facts bundle

5.3 Example Commands

```
# Quick table view of all deployments
poetry run piqc scan --format table

# Detailed YAML for specific namespace
poetry run piqc scan --namespace prod --format yaml

# Full scan with runtime metrics
poetry run piqc scan --namespace inference \
  --collect-runtime \
  --format yaml \
  -o ./results

# Fast scan without GPU metrics
poetry run piqc scan --no-exec --format json

# Debug mode for troubleshooting
poetry run piqc scan --namespace test --debug

# Generate ParallelIQ facts bundle
poetry run piqc scan --namespace inference \
  --output-piqc \
  -o ./piqc-facts
```

6 Output Formats

6.1 Table Format

Quick visual overview of discovered deployments:

```
  Model Name              Engine     GPU Type       Replicas     GPU Util

  TinyLlama-1.1B-...      vllm       1xTesla T4           1          15%
```

6.2 YAML Format

Complete ModelSpec with all collected data:

```yaml
apiVersion: modelspec.paralleliq.ai/v1
kind: ModelSpec
metadata:
  name: vllm-tinyllama
  namespace: inference
model:
  name: TinyLlama/TinyLlama-1.1B-Chat-v1.0
  architecture: llama
  parameters: 1.1B
resources:
  gpus:
  - type: Tesla T4
    memoryTotal: 15GB
    temperature: 55
runtimeState:
  vllm:
    loadedModel: tinyllama
    healthStatus: healthy
    promptTokensPerSec: 1250.5
```

6.3 JSON Format

Machine-readable format for automation:

```json
{
  "apiVersion": "modelspec.paralleliq.ai/v1",
  "kind": "ModelSpec",
  "metadata": {
    "name": "vllm-tinyllama",
    "namespace": "inference"
  },
  "model": {
    "name": "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
  }
}
```

6.4 PIQC Facts Bundle

Standardized facts format for ParallelIQ platform:

```json
{
  "schema": "piqc-scan.v0.1",
  "workloadObjects": [
    {
      "id": "inference/vllm-tinyllama",
      "facts": {
        "runtime.engineType": "vllm",
        "model.name": "TinyLlama-1.1B-Chat-v1.0",
        "hardware.gpuType": "Tesla T4"
      }
    }
  ]
}
```

7 Troubleshooting

7.1 Common Issues

7.1.1 Connection Errors

Error: `Unable to connect to cluster`
Solution:

```
# Verify kubectl works
kubectl get nodes

# Check current context
kubectl config current-context

# Switch context if needed
kubectl config use-context my-cluster
```

7.1.2 RBAC Permission Errors

Error: `User cannot list pods in namespace`
Solution:

```
# Apply RBAC permissions
kubectl apply -f rbac/

# Or ask admin to grant cluster-viewer role
```

7.1.3 GPU Metrics Not Available

Error: `nvidia-smi: command not found`
Solution:

- Install GPU drivers in cluster
- Or use `-no-exec` flag to skip GPU metrics

7.1.4 Runtime Metrics Not Collected

Error: `Could not discover vLLM service endpoint`
Solution:

- Verify vLLM pod is running: `kubectl get pods -n <namespace>`
- Check vLLM is serving on port 8000
- Ensure RBAC allows pod exec

7.2 Debug Mode

For detailed troubleshooting:

```
poetry run piqc scan --namespace inference --debug
```

7.3 Getting Help

- GitHub Issues: https://github.com/your-org/ModelSpec/issues
- Documentation: https://your-org.github.io/ModelSpec
- Email: support@paralleliq.ai

8 Best Practices

1. **Regular Scanning**: Run PIQC scans regularly to track deployment changes
2. **Use Namespaces**: Scan specific namespaces for faster results
3. **Collect Runtime Metrics**: Use `-collect-runtime` for production monitoring
4. **Save Outputs**: Always use `-o` flag to save results
5. **Version Control**: Store ModelSpec files in version control
6. **Automate**: Integrate PIQC into CI/CD pipelines